

# TRENTOS University Course (@ TUM) - WS2022 - Team Projects

## Selected project scope

### Team 1 - Drone Simulator (3 students per team)

- Project description
  - A main application running on a TRENTOS instance on the RPi 3 B+ shall
    - control a drone in a drone simulator that runs on a Linux PC and
    - initiate a typical flight consisting of start, flying phase and landing
  - Relevant sensor data is exchanged between TRENTOS and the PC-based drone simulator
    - via Ethernet and
    - by using the API provided by the simulator
  - Relevant actuator data is exchanged between TRENTOS and a PX4 flight controller
    - via Ethernet between RPi 3 B+ and Linux PC and via USB (serial) between Linux PC and flight controller
    - by using the MAVLink protocol
- Individual tasks
  - Connection of a drone simulator (Gazebo) environment for controlling a virtual drone via TRENTOS in order to
    - start the drone
    - initiate the drone flight by continuously
      - receiving sensor data from the simulator
      - calculating required actuator control commands
      - sending the control commands to a PX4 flight controller
    - land the drone
  - Setting up the PX4 flight controller Software in the Loop (SITL) case
  - Setting up the PX4 flight controller Hardware in the Loop (HITL) case

### Teams 2 & 3 - TPM - Encrypted File System (3 students per team)

- Project description
  - A "Secure Storage" component, running on a TRENTOS instance on the RPi 3 B+, shall be developed
  - It should utilize a hardware-based TPM device, by means of a "TPM Driver" component, which
    - provides a source of entropy via a TPM-based random number generator (RNG)
    - provides access to a TPM-based key store
    - applies hardware-accelerated crypto functionality
  - The "Secure Storage" component shall be usable by a main application
    - in order to provide an encrypted storage backend
    - accessible via the TRENTOS FileSystem API
    - while encryption/decryption of data is transparently handled by the "Secure Storage" component
- Individual tasks
  - A purely software-based solution for the "Secure Storage" component shall be implemented, utilizing
    - the TRENTOS Crypto API (for encryption/decryption)
    - the TRENTOS KeyStore API (for securely loading/storing keys)
    - the TRENTOS **EntropySource** component (as source of entropy)
    - the TRENTOS **RamDisk** component (as raw storage "backend")
  - Connection of a SPI-based TPM device and development of the "TPM Driver" component
    - a LetsTrust TPM breakout board is used
    - an existing driver/library can be ported exemplarily to TRENTOS
  - Integration of the "TPM Driver" into the software-based solution
  - Measure performance difference between purely software-based and TPM-based solution

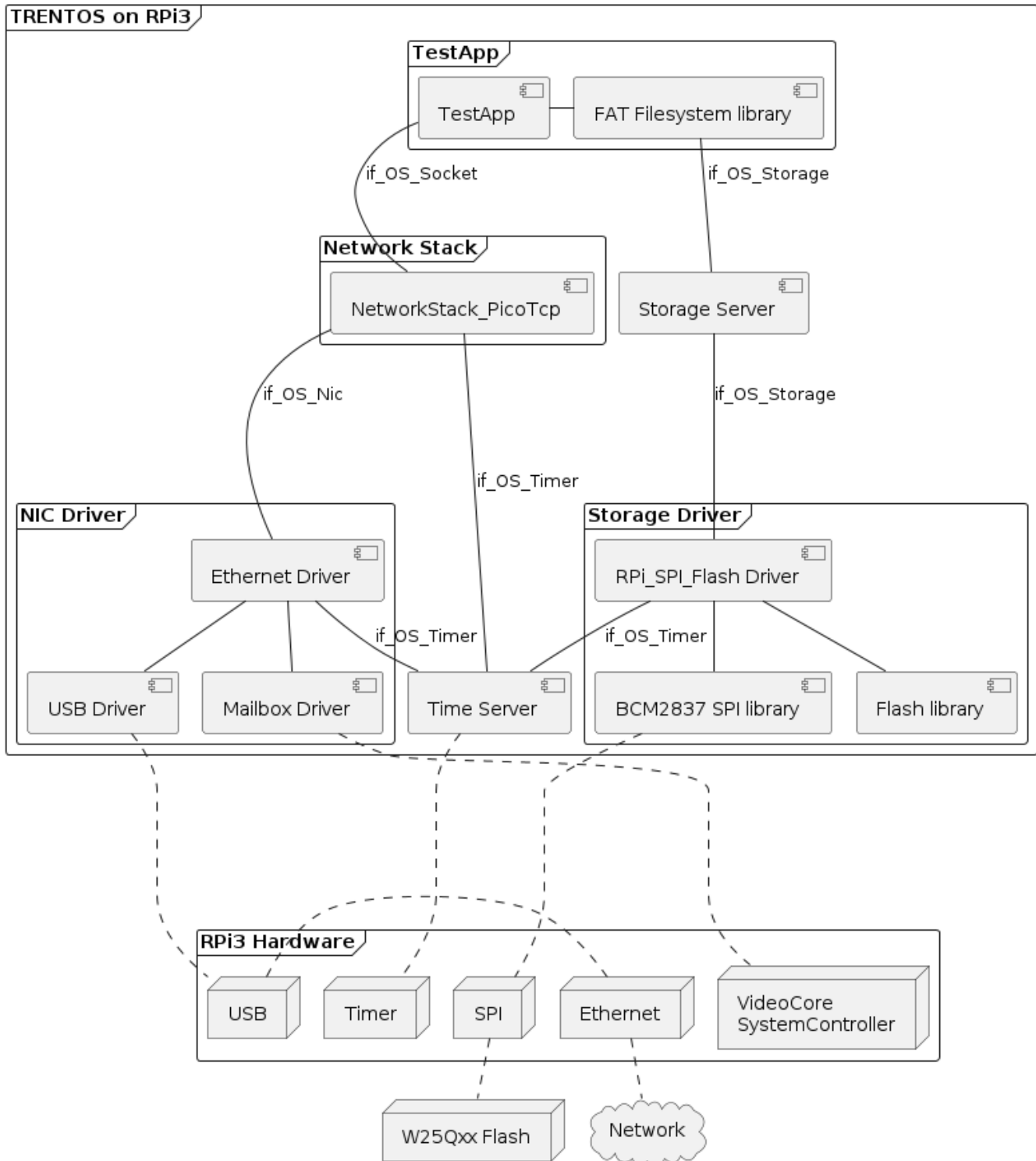
### Team 4 - TPM - Secure Communication (2 students per team)

- Project description
  - A "Secure Communication" component, running on a TRENTOS instance on the RPi 3 B+, shall be developed
  - It should utilize a hardware-based TPM device, by means of a "TPM Driver" component, which
    - provides a source of entropy via a TPM-based random number generator (RNG)
    - provides access to a TPM-based key store
    - applies hardware-accelerated crypto functionality
  - The "Secure Communication" component shall be usable by a main application
    - in order to communicate with a remote server
    - while encryption/decryption of messages is transparently handled by the "Secure Communication" component
- Individual tasks
  - A purely software-based solution for the "Secure Communication" component shall be implemented, utilizing
    - the TRENTOS Crypto API (for encryption/decryption)
    - the TRENTOS KeyStore API (for securely loading/storing keys)
    - the TRENTOS **EntropySource** component (as source of entropy)
  - Connection of a SPI-based TPM device and development of the "TPM Driver" component
    - a LetsTrust TPM breakout board is used
    - an existing driver/library can be ported exemplarily to TRENTOS
  - Integration of the "TPM Driver" into the software-based solution

- Measure performance difference between purely software-based and TPM-based solution

## Base Case

Throughout the homework assignments, we have learned all the basics regarding dealing with storage and networking on top of TRENTOS. Let's have a look at the final setup description from Homework Task 5 again:



### Initial System Architecture

This system will be our basic setup to start with, regardless of which specific team project task is assigned to you. As already seen before, we can split our initial system into three different logical parts:

- the actual application, which contains our own program logic
- the networking part, consisting of a network driver and the network stack
- the storage part, consisting of a storage driver and the storage server

For the project tasks, we now have to concentrate on three main aspects:

- for the drone simulator case, we want to
  - use the initial setup and extend the networking part (as client) in order to communicate with an external simulator (which is acting as a server)
  - add a connection to a "simulated" flight controller board
  - add a connection to a real flight controller board (based on the [Diatone Mamba F405](#))
- for the TPM case, we want to
  - use the initial setup and extend the storage part by both software-based and hardware-accelerated encryption/decryption functionality or
  - use the initial setup and extend the networking part by both software-based and hardware-accelerated encryption/decryption functionality
  - add a new driver for a SPI-based TPM breakout board (based on the [Infineon OPTIGA SLB9670 TPM 2.0](#))

As implementing a new driver from scratch would be too challenging by now, we will concentrate on re-using existing open source drivers (and libraries) and porting them onto TRENTOS. More information will be provided in the sections below.

## Project Tasks

For each of the project tasks described above more details regarding the implementation will be provided in the following sections.

### Drone Simulator (Team 1)

The application of simulators allows for a quick, easy and safe way of testing drones. The simulation thereby provides a respective 3D environment as well as a 3D model of a drone that is able to interact with the environment with the help of a set of sensors and actuators. Respective sensor data has to be processed by dedicated computing nodes, which calculate corresponding actuator control commands. In general, two types of computing nodes exist:

- a small flight controller, usually based on an ARM Cortex-M platform, which has access to a set of sensors and which can control the drone's actuators
- a companion (or mission) computer, usually based on an ARM Cortex-A platform, which has access to more sophisticated sensors (e.g. a camera) and which provides the required computing power for processing corresponding data (e.g. as part of AI related use cases)

A typical drone architecture that makes use of both flight controller and companion computer is described at [https://docs.px4.io/main/en/concept/px4\\_systems\\_architecture.html#fc-and-companion-computer](https://docs.px4.io/main/en/concept/px4_systems_architecture.html#fc-and-companion-computer). As part of this project, a virtual drone setup shall be implemented that mimics the setup of a regular physical drone. Both types of computing nodes shall be utilized and exemplarily be connected to the Gazebo simulator, in order to provide a small drone flight scenario. Thereby, the virtual drone setup can be split into two variants:

- a Software in the Loop (SITL) setup, where the flight stack (here: based on PX4) of the flight controller runs on a PC (either the same PC as the Gazebo simulation or another PC on the same network) instead of being executed on a real flight controller board
- a Hardware in the Loop (HITL) setup, where the flight stack (here: based on PX4) runs on a real flight controller board, but has only access to a "simulation firmware" that restricts access to actual physical sensors or actuators

Within the virtual drone setup, in both cases, sensors (e.g. GPS) and actuators (e.g. motors) are provided only in virtual form via the coupled simulator Gazebo. The interplay of simulator, flight controller and companion computer shall finally allow for the realization of a small drone flight scenario. This flight scenario looks like the following: the drone shall

- take off
- reach a predefined height (e.g. by using a distance sensor)
- start flying a predefined distance (e.g. by using a GPS sensor)
- stop
- land on the ground

As part of the project, this flight scenario shall be realized with the help of both the SITL and the HITL approach.

### Case 1 - Software in the Loop (SITL)

As part of the SITL case,

- a TRENTOS-based companion computer (utilizing the RPi 3 B+)
- a flight controller using the PX4 stack (provided via SITL)
- a drone simulator (in form of Gazebo)

have to be provided. A typical SITL setup is described at <https://docs.px4.io/main/en/simulation/#sitl-simulation-environment>. Hereby,

- the "PX4 on SITL" part represents the PX4 flight controller
- the "API/Offboard" part represents our TRENTOS-based companion computer
- the "Simulator" part represents Gazebo

As can be seen within the figure provided via the link above, on a physical drone, these individual parts have to communicate in order to control a respective flight. For being able to exchange data, the individual parts therefore have to be connected respectively. Some parts hereby rely on a standard protocol, which is called **MAVLink** (<https://mavlink.io/en/>). Others may require some kind of custom protocol. In order to realize the described flight scenario,

- the TRENTOS companion computer shall continuously receive sensor data from the Gazebo simulator during the

- take off phase, utilizing a distance sensor, in order to
    - measure the reached height of the drone at its current position
    - compare the measured height to a predefined threshold value
    - initiate flight phase, in case the threshold is reached
  - the flight phase, utilizing a GPS sensor, in order to
    - calculate the currently covered distance, by combining the newly received data with previously captured data
    - compare the calculated distance to a predefined threshold value
    - initiate a stop, in case the threshold is reached
- the TRENTOS companion computer shall send control data to the PX4 flight controller, in order to
  - start the drone's motors
  - initiate drone take off
  - start the flight phase
  - stop the flight phase
  - initiate drone landing
  - stop the drone's motors
- the PX4 flight controller shall communicate with Gazebo, in order to visualize the desired behavior (e.g. motors turning, drone actually moving)

Communication between the PX4 flight controller and Gazebo as well as between the PX4 flight controller and the TRENTOS companion computer is realized by utilizing the MAVLink protocol. Gazebo thereby communicates with PX4 by using the Simulator MAVLink API (<https://docs.px4.io/main/en/simulation/#simulator-mavlink-api>). For TRENTOS, MAVLink protocol support is currently missing and thus has to be provided as part of the to be developed TRENTOS application. Hereby, the **mavgen** generator (<https://mavlink.io/en/#mavlink-project-generatorslanguages>) can be used to generate the required MAVLink libraries for various programming languages. In order to support TRENTOS, support for the C programming language has to be selected. On the Linux PC, a MAVLink router application like **mavp2p** (<https://github.com/aler9/mavp2p>) or **mavlink-router** (<https://github.com/mavlink-router/mavlink-router>) can then be used to route MAVLink encoded data between the respective MAVLink nodes.

Communication between the TRENTOS companion computer and Gazebo for exchanging sensor data is based on a custom connection. Gazebo hereby provides a TCP/IP socket communication, utilizing Google Protobuf for message passing and serialization. This requires the development of a small C++ proxy application, which runs on the Linux PC and which

- receives sensor data (here: data from a distance sensor and a GPS sensor) from Gazebo by using TCP/IP sockets, thereby utilizing Google Protobuf for message passing and serialization
- deserializes Protobuf data and converts it into a format to be used by the TRENTOS application
- sends the converted data to TRENTOS (utilizing an Ethernet-based network connection) by using TCP/IP sockets

Examples that demonstrate the utilization of Gazebo's socket-based transport can be found here

- Gazebo Classic
  - [https://classic.gazebosim.org/tutorials?tut=topics\\_subscribed&cat=transport](https://classic.gazebosim.org/tutorials?tut=topics_subscribed&cat=transport)
  - [https://classic.gazebosim.org/tutorials?tut=custom\\_messages&cat=transport](https://classic.gazebosim.org/tutorials?tut=custom_messages&cat=transport)
- Gazebo
  - <https://gazebosim.org/api/transport/11.0/tutorials.html>
  - <https://github.com/gazebosim/gz-transport>

## Case 2 - Hardware in the Loop (HITL)

As part of the HITL case,

- a TRENTOS-based companion computer (utilizing the RPi 3 B+)
- a flight controller using the PX4 stack (provided via HITL, exemplarily using the [Diatone Mamba F405](#))
- a drone simulator (in form of Gazebo)

have to be provided. A typical HITL setup is described at <https://docs.px4.io/main/en/simulation/hitl.html#hitl-simulation-environment>. Hereby,

- the "PX4 HITL" part represents the PX4 flight controller
- the "API/Offboard" part represents our TRENTOS-based companion computer
- the "Simulator" part represents Gazebo

As can be seen within the figure provided via the link above, analogous to the SITL case, these individual parts have to communicate in order to control a respective flight. For being able to exchange data, the individual parts therefore also have to be connected respectively. As the only difference to the SITL setup is provided in form of the real flight controller device, which replaces the SITL part, most of the implementation can be reused. Nevertheless,

- the real flight controller device (here: the Diatone Mamba F405) must be connected to the Linux PC (via USB)
- the existing MAVLink communication configuration (using **mavp2p** or similar) has to be adapted in order to also allow routing messages to a serial interface

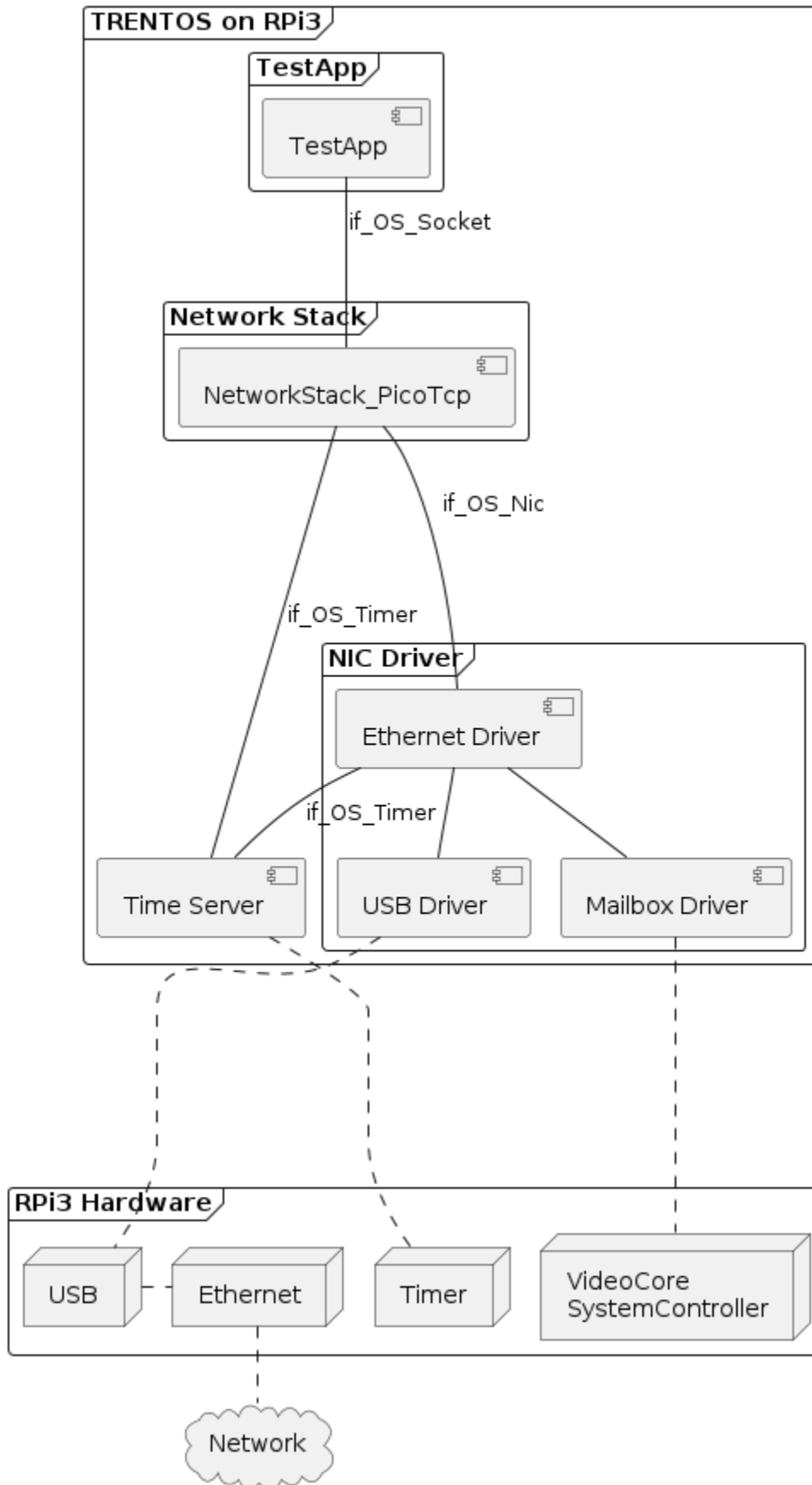
## What to do

In order to realize the presented SITL and HITL cases, the following steps might provide a helpful guideline:

1. Install the required simulator environment
  - a. Provide the underlying hardware/software test setup, consisting of
    - i. a Linux PC (as host) running Gazebo and the flight controller SITL
    - ii. a TRENTOS instance running on the RPi 3 B+, which is connected to the Linux PC via Ethernet
  - b. Get familiar with the overall PX4 toolchain installation: [https://docs.px4.io/main/en/dev\\_setup/dev\\_env\\_linux\\_ubuntu.html#gazebo-jmavsim-and-nuttx-pixhawk-targets](https://docs.px4.io/main/en/dev_setup/dev_env_linux_ubuntu.html#gazebo-jmavsim-and-nuttx-pixhawk-targets)
    - i. PX4
    - ii. Gazebo 9
  - c. Get familiar with the Gazebo simulator environment

- Gazebo: <https://gazebo-sim.org/home>
  - Gazebo Classic: <https://classic.gazebo-sim.org/>
- d. Get familiar with the PX4 Autopilot (<https://docs.px4.io/main/en/>)
  2. Provide the basic connection between flight controller SITL and Gazebo
    - a. Understand how to connect Gazebo and PX4 via MAVLink using the SITL approach
      - i. select a world (e.g. empty.world)
      - ii. select a drone model (e.g. basic quadcopter)
      - iii. understand general information that can be found at
        1. <https://docs.px4.io/main/en/simulation/gazebo.html>
        2. <https://docs.px4.io/main/en/simulation/#starting-building-sitl-simulation>
    - b. Understand how to create a small PX4 application that is able to receive control commands from an external source (e.g. Gazebo or the companion computer) in order to control the drone's motors
      - i. [https://docs.px4.io/main/en/dev\\_setup/building\\_px4.html](https://docs.px4.io/main/en/dev_setup/building_px4.html)
      - ii. [https://docs.px4.io/main/en/modules/hello\\_sky.html](https://docs.px4.io/main/en/modules/hello_sky.html)
    - c. Implement the "basic" flight scenario for starting, flying and landing the drone within a dedicated PX4 application
      - i. provide functionality to
        1. start the drone's motors
        2. initiate drone take off
        3. start the flight phase
        4. fly for some time (e.g. a few seconds)
        5. stop the flight phase
        6. initiate drone landing
        7. stop the drone's motors
      - ii. exemplarily receive sensor data (e.g. from a GPS sensor) from Gazebo
  3. Extend the flight controller SITL and Gazebo setup via a TRENTOS-based companion computer
    - a. Reuse the initial TRENTOS setup provided within section "Base Case"
    - b. Remove the following parts from the existing setup
      - i. TRENTOS Filesystem API utilization from **TestApp**
      - ii. TRENTOS **StorageServer** component utilization
      - iii. TRENTOS **RPi\_SPI\_Flash** component utilization
    - c. Extend the existing TRENTOS **TestApp** to send control data to the flight controller SITL
      - i. use the **mavgen** generator (<https://mavlink.io/en/#mavlink-project-generatorslanguages>) to generate the required MAVLink libraries for the C programming language
      - ii. provide support for the MAVLink protocol in TRENTOS by integrating the generated MAVLink libraries
      - iii. prepare the respective TRENTOS application logic for generating control commands that are required by the flight controller SITL for starting, flying and landing
        1. HINT 1: transfer relevant parts of the existing flight control logic from your PX4 application to the **TestApp**
        2. HINT 2: use dummy data at this phase, to test the principal connection between TRENTOS and flight controller SITL (e.g. the drone starts, flies for 10 seconds and then lands on the ground)
    - d. Integrate the TRENTOS instance into the existing MAVLink-based communication setup
      - i. analyze and understand the existing MAVLink communication between flight controller SITL and Gazebo
      - ii. apply a MAVLink router application like **mavp2p** (<https://github.com/aler9/mavp2p>) in order to route MAVLink encoded data between the respective MAVLink nodes
  4. Provide a dedicated connection between TRENTOS and Gazebo for exchanging sensor data
    - a. Equip the drone in Gazebo with additional sensors (e.g. consulting <https://www.youtube.com/watch?v=0zQ8iFgw6uk>)
      - i. a distance sensor (for measuring distance to the ground)
      - ii. a GPS sensor
    - b. Provide a C++ proxy application that
      - i. runs on the Linux PC
      - ii. extracts relevant sensor data (from a distance sensor and a GPS sensor) from Gazebo
      - iii. provides the translation between Gazebo and the socket-based communication required by TRENTOS
    - c. Extend the existing TRENTOS **TestApp** via a socket-based connection to the C++ proxy application in order to
      - i. receive data from the newly added sensors (from a distance sensor and a GPS sensor) in Gazebo
      - ii. provide the required measurement and calculations based on the received sensor data, and depending on the respective phase (either take off or flight)
    - d. Adapt the existing TRENTOS **TestApp** to send calculated actuator control data to the flight controller by using the previously integrated MAVLink protocol support
      - i. adapt/extend the already integrated application logic for generating flight control commands where required
      - ii. replace the dummy data from step 3a) with actual actuator data calculated based on the received sensor data
  5. Replace the flight controller SITL by a flight controller HITL setup
    - a. Get familiar with the PX4 HITL simulation (<https://docs.px4.io/main/en/simulation/hitl.html>)
    - b. Apply the Diatone Mamba F405 as flight controller (which is supported in PX4 since **Stable Release v1.13.0**)
      - i. get familiar with the flight controller device (<https://www.multirotorguide.com/news/diatone-mamba-f405-and-f405-mini-stacks/>)
      - ii. connect the device via microUSB-to-USB cable to the Linux PC
      - iii. use **QGroundControl** (<http://qgroundcontrol.com/>) to
        1. flash the PX4 software
          - a. <https://docs.qgroundcontrol.com/master/en/SetupView/Firmware.html>
          - b. <https://docs.px4.io/main/en/config/firmware.html>
        2. configure the flight controller as HITL device and and arm/disarm it respectively (<https://docs.px4.io/main/en/simulation/hitl.html#setting-up-hitl>)
        3. configure the simulator-specific setup (<https://docs.px4.io/main/en/simulation/hitl.html#simulator-specific-setup>)
    - c. Install the existing PX4 application on the new flight controller
    - d. Integrate the flight controller HITL (as serial device) into the existing **mavp2p** (or similar) MAVLink router setup
    - e. Run the existing virtual drone setup, consisting of
      - i. a Linux PC (as host) running Gazebo
      - ii. the Diatone Mamba F405, which is connected to the Linux PC via USB, acting as flight controller and running the PX4 flight control stack as well as the developed PX4 application
      - iii. a TRENTOS instance running on the RPi 3 B+, which is connected to the Linux PC via Ethernet

The final simulator-based solution shall look like the following:



# TPM

The respective initial setup (demonstrated in section "Base Case") we want to use for porting the SPI-based TPM breakout board has a small limitation. TRENTOS currently only supports one SPI-based peripheral at the same time. As already described in the project scope at the beginning, a main application (**TestApp**) running on a TRENTOS instance on the RPi 3 B+ shall utilize a hardware-based TPM device to provide access to a TPM-based key store, a TPM-based Random Number Generator (RNG) and apply hardware-accelerated crypto functionality (in contrast to the existing software-based TRENTOS Crypto API available in the SDK). The existing TRENTOS system setup from Homework Task 5 shall be re-used as starting point.

## TPM - Encrypted File System (Teams 2 & 3)

Up to this point, data was stored unencrypted either on a RamDisk or a SPI NOR flash with the help of the TRENTOS FileSystem API. As part of this project, we want to change this and provide a protected storage for the data. We utilize a "full disk encryption" approach and therefore encrypt the whole "disk". For this, we make use of the component-based architecture of TRENTOS and provide our own "Secure Storage" component, which exposes the **if\_OS\_Storage** interface to other components. As only one SPI-based peripheral can be used at the same time, we have to switch from the currently used TRENTOS **RPI\_SPI\_Flash** component back to the TRENTOS **RamDisk** component (TRENTOS Handbook, chapter "TRENTOS RamDisk").

The "Secure Storage" component itself receives **write/read** calls from an application and

- encrypts/decrypts the data before writing/reading to the actual storage
- forwards the encrypted data to and the decrypted data from the RamDisk storage "backend"

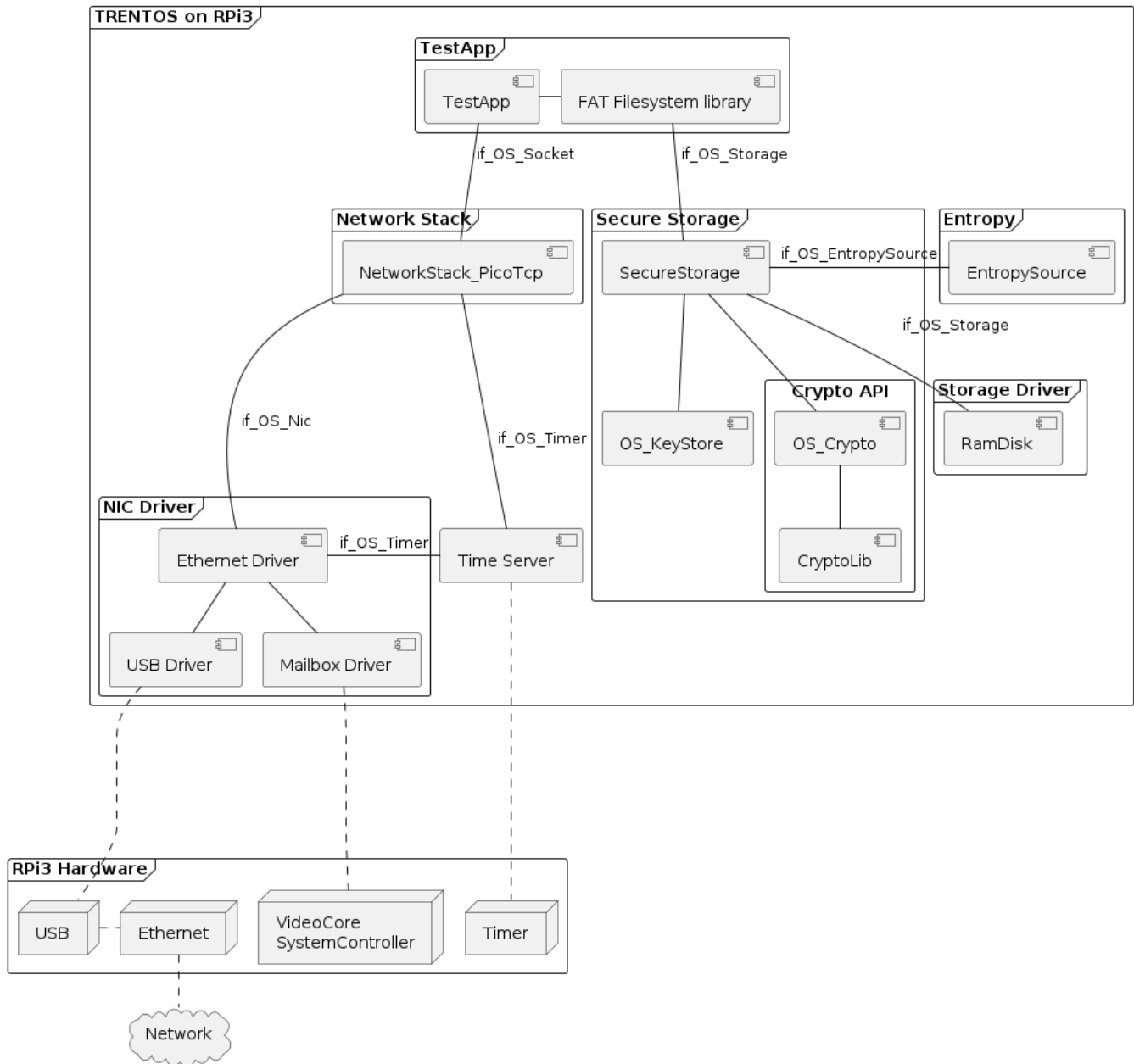
This way, the application itself can access the RamDisk-backed file system by using the TRENTOS FileSystem API, while the "Secure Storage" component transparently handles all the cryptographic operations.

## Software-based Solution

To allow working in parallel, and in order to establish a functional basis, we first want to develop a purely software-based solution. The following steps shall therefore provide a reference:

- Development of a "Secure Storage" component
  - Shall implement the **if\_OS\_Storage** interface for usage by other components
  - Respective **write/read** calls need to be implemented
    - On write access
      - Read the respective block from RamDisk via the **if\_OS\_Storage** interface of the TRENTOS **RamDisk** component
      - Decrypt the block
      - Write data to the block
      - Encrypt the block
      - Write the block back to RamDisk
    - On read access
      - Read the respective block from RamDisk
      - Decrypt the block
      - Return the data
  - Keys shall be stored and loaded via the TRENTOS KeyStore API
  - Encryption/Decryption of data shall be done via the TRENTOS Crypto API
  - As entropy source, the TRENTOS **EntropySource** component shall be used
- Replace the TRENTOS **RPI\_SPI\_Flash** component by the TRENTOS **RamDisk** component
- Remove the TRENTOS **StorageServer** component utilization
- Extension of the **TestApp** component in order to
  - utilize the TRENTOS FileSystem API, using the FAT file system type
  - make use of the "Secure Storage" component via the **if\_OS\_Storage** interface
  - reuse the existing networking capabilities (networking as client) introduced as part of Homework Task 5
  - receive the content delivered by the webserver introduced as part of Homework Task 5
  - encrypt the received content
  - store the encrypted content on the RamDisk using the FAT file system
  - read back the encrypted content, decrypt it and visualize the data on the console

The final software-based solution shall look like the following:



## RPi Secure Storage - Architecture

### TPM-based Solution

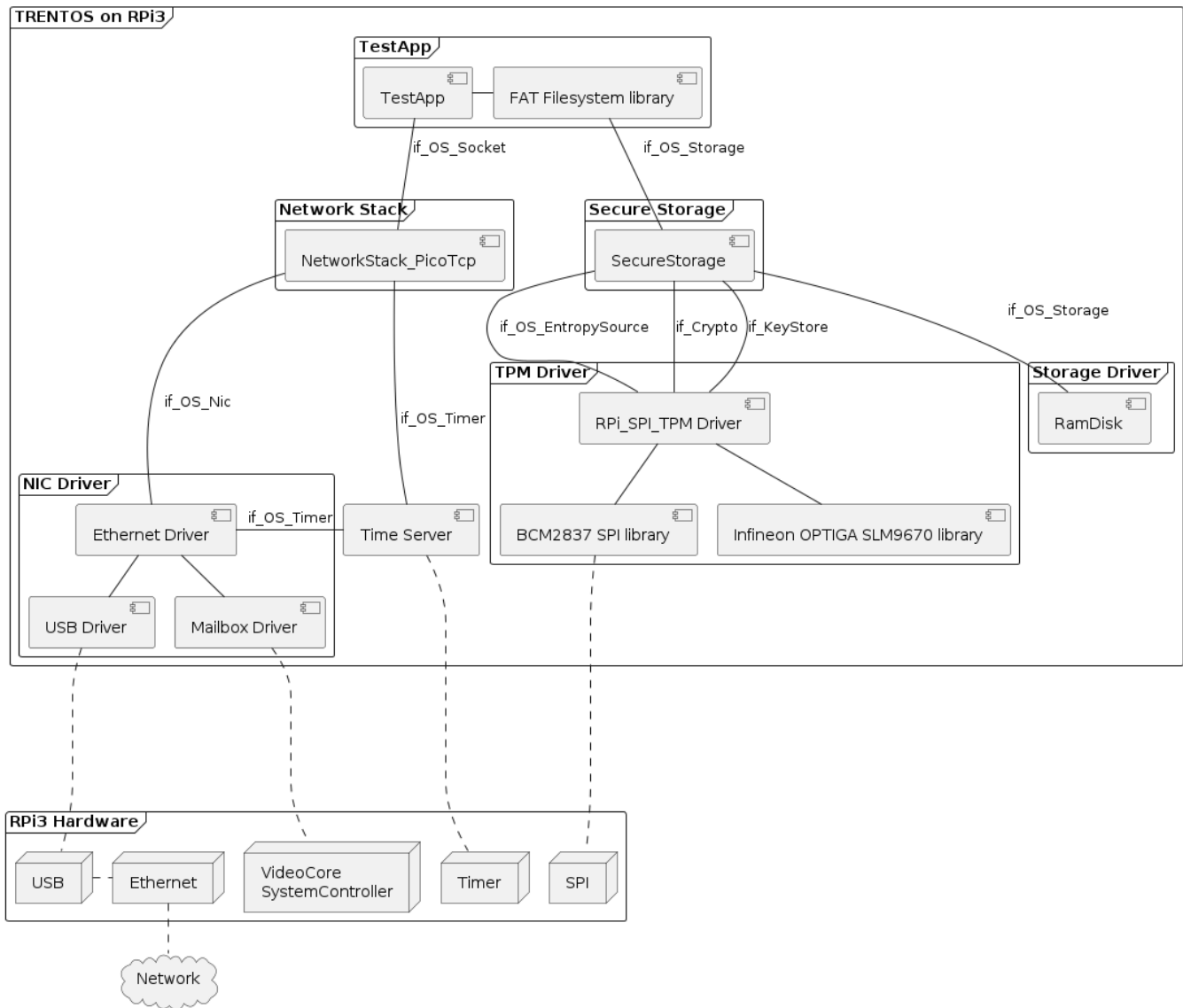
For a successful driver development and integration of the LetsTrust TPM breakout board into our system, the following steps shall provide a reference:

1. As already mentioned before, we can only support one SPI-based peripheral for now on the RPi 3 B+. Hence, analogous to the software-based solution, we work with the TRENTOS **RamDisk** component (TRENTOS Handbook, chapter "TRENTOS RamDisk") instead of the TRENTOS **RPi\_SPI\_Flash** driver component, in order to start with the initial setup.
2. Analogous to the software-based solution, we also have to remove the TRENTOS **StorageServer** component utilization.
3. Afterwards, we have to provide the correct wiring of the SPI-based TPM breakout board to the RPi 3 B+. Please use the wiring of the SPI NOR Flash described in the TRENTOS Handbook, chapter "TRENTOS RPi\_SPI\_Flash" as reference.
4. Nevertheless, the driver internals still have to be adapted to the new Infineon OPTIGA SLB9670 TPM 2.0 based hardware device. As we want to provide support for a SPI-based peripheral, please have a look at the TRENTOS **RPi\_SPI\_Flash** component (see TRENTOS Handbook, chapter "TRENTOS RPi\_SPI\_Flash"). It can be considered as a starting point regarding the driver implementation for SPI-based peripherals in general, as it already provides the required abstractions and support for the respective low-level mechanics:
  - a. the GPIO handling (see `bcm2837_gpio.h` and `bcm2837_gpio.c`)
  - b. the SPI protocol (see `bcm2837_spi.h` and `bcm2837_spi.c`)
5. On top of the GPIO and SPI abstractions, we then have to provide the actual peripheral-specific functionality. Within the TRENTOS **RPi\_SPI\_Flash** component, this is wrapped into a peripheral-specific library, e.g. here for using a flash storage on top of the SPI protocol (see `spiflash.h` and `spiflash.c`). We can do the same for our SPI-based TPM peripheral (see section "Potential resources" below for more information). It is recommended to have a look at the open-source TCG TPM2 Software Stack and the open-source wolfTPM library.
6. Finally, we have to create a TRENTOS specific wrapper on top of this which represents the actual "driver" component within the OS (for the TRENTOS **RPi\_SPI\_Flash** component: `RPi_SPI_Flash.c` and `RPi_SPI_Flash.camkes`).



7. Examine both the mbedTLS library (which we use within the TRENTOS Crypto API) and the Infineon OPTIGA SLB9670 TPM 2.0 to select a cryptographic algorithm that is supported by both variants, e.g. the RSA2048.
8. Development of the "TPM Driver" component
  - a. Expose the TPM's RNG via the **if\_OS\_EntropySource** interface
  - b. Provide your own interface (**if\_KeyStore**) for loading/storing keys
  - c. Provide your own interface (**if\_Crypto**) for generating keys and encryption/decryption of data
9. Step by step migrate the software-based solution to the TPM-based solution
  - a. Replace the TRENTOS **EntropySource** component by connecting to the **if\_OS\_EntropySource** interface of the "TPM Driver"
  - b. Replace the utilization of the TRENTOS KeyStore API by using respective TPM-based functions offered via the **if\_KeyStore** interface of the "TPM Driver"
  - c. Replace the utilization of the TRENTOS Crypto API by using respective TPM-based functions offered via the **if\_Crypto** interface of the "TPM Driver"

The final TPM-based solution shall look like the following:



## RPi TPM-based Secure Storage - Architecture

## Evaluation

We want to evaluate both solutions (software-based and TPM-based) and compare them according to their performance. Therefore we want to measure the performance of the following operations:

1. Initialization of the file system
2. Reading a file from the file system
3. Writing a file to the file system

## Potential Resources

The SPI-based TPM breakout board we will utilize for the project task is based on the Infineon OPTIGA SLB9670 TPM 2.0. Please have a look at the following resources, which provide helpful information and source code examples for driver/library development based on this specific chipset. These resources can be used as a starting point for implementing (or rather porting) your own driver/library to TRENTOS. Note: don't remove any license related information (e.g. GPL statements) and document from which sources you have taken code or code snippets for your own driver!

- <https://letstrust.de/>
- <https://www.infineon.com/cms/de/product/security-smart-card-solutions/optiga-embedded-security-solutions/optiga-tpm/slm-9670/>
- <https://www.infineon.com/cms/de/product/security-smart-card-solutions/optiga-embedded-security-solutions/optiga-tpm/slb-9670vq2.0/#documents>
- [https://www.infineon.com/dgdl/Infineon-SLM%209670-DataSheet-v01\\_00-EN.pdf?fileId=5546d46269e1c019016a21819cc80c93](https://www.infineon.com/dgdl/Infineon-SLM%209670-DataSheet-v01_00-EN.pdf?fileId=5546d46269e1c019016a21819cc80c93)
- [https://www.infineon.com/dgdl/Infineon-product-brief-optiga-tpm9670-PB-v01\\_00-EN.pdf?fileId=5546d462696dbf1201697c89f4bb4789](https://www.infineon.com/dgdl/Infineon-product-brief-optiga-tpm9670-PB-v01_00-EN.pdf?fileId=5546d462696dbf1201697c89f4bb4789)
- [https://www.infineon.com/dgdl/Infineon-App-Note-SLx9670-TPM2.0\\_Embedded\\_RPi\\_DI\\_SLx-ApplicationNotes-v01\\_03-EN.pdf?fileId=5546d46267c74c9a01684b96e69f5d7b](https://www.infineon.com/dgdl/Infineon-App-Note-SLx9670-TPM2.0_Embedded_RPi_DI_SLx-ApplicationNotes-v01_03-EN.pdf?fileId=5546d46267c74c9a01684b96e69f5d7b)
- <https://www.infineon.com/cms/de/product/evaluation-boards/iridium-slm-9670-tpm2.0/>
- <https://github.com/tpm2-software/tpm2-tss>
- <https://github.com/wolfssl/wolfTPM>
- [https://ubs\\_csse.gitlab.io/secu\\_os/tutorials/tpm\\_rpi.html](https://ubs_csse.gitlab.io/secu_os/tutorials/tpm_rpi.html)

## Things to watch out for

- By default, the LetsTrust TPM uses SPI 0, chip select 1
- The RPi GPIO Pin 18 needs to be pulled high
- No symmetric encryption is supported by our TPM

## TPM - Secure Communication (Team 4)

Up to this point, messages were transmitted unencrypted via the TRENTOS Socket API. As part of this project, we want to change this and provide secure socket communication for messages. We therefore make use of the component-based architecture of TRENTOS and provide our own "Secure Communication" component, which exposes the **if\_OS\_Socket** interface to other components. The "Secure Communication" component itself receives **write/recvfrom** calls from an application and

- encrypts/decrypts the messages before writing/receiving
- forwards the encrypted messages to and the decrypted messages from the network stack.

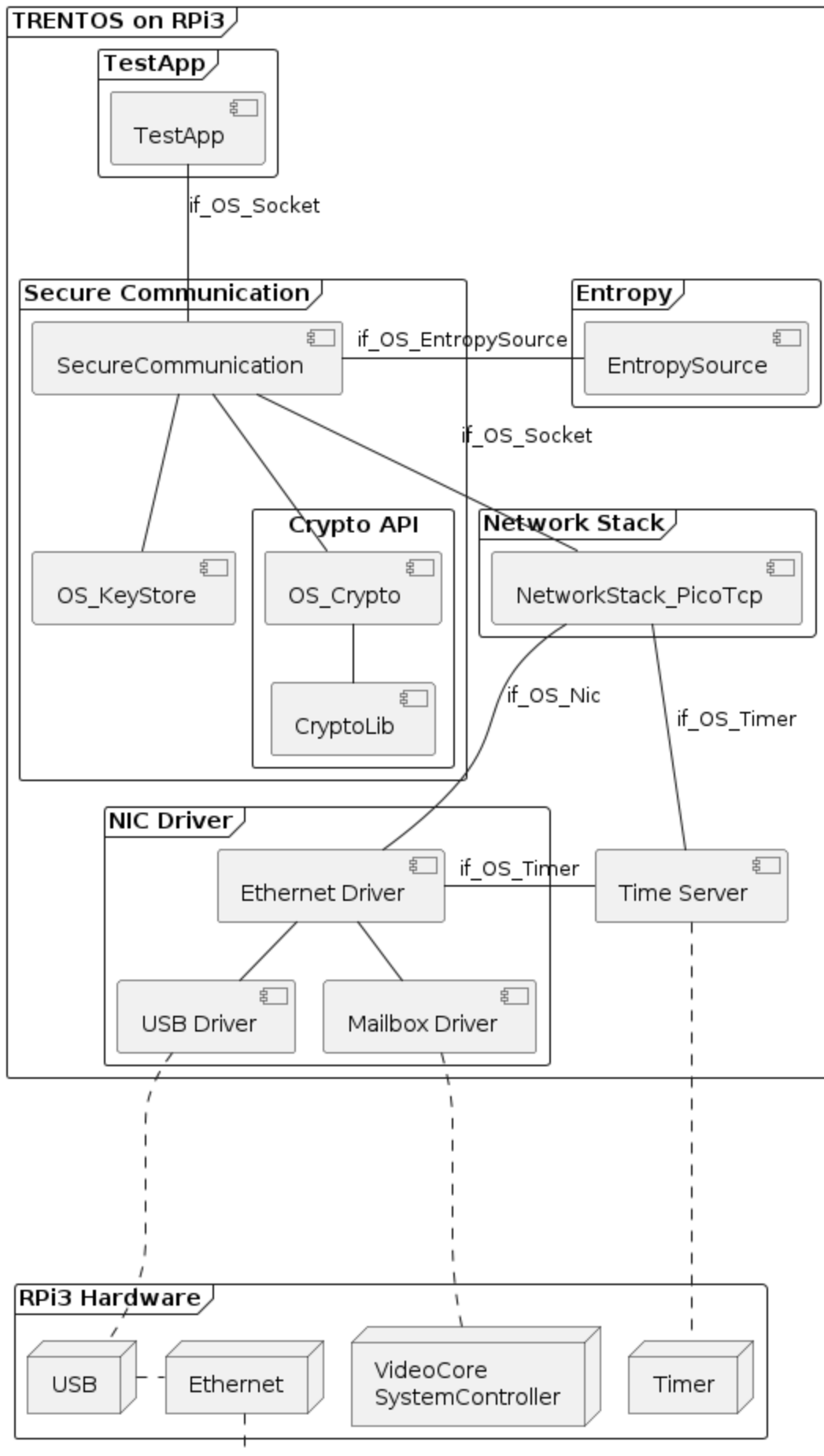
This way, the application itself can communicate with a remote party by using the TRENTOS Socket API, while the "Secure Communication" component transparently handles all the cryptographic operations.

## Software-based Solution

To allow working in parallel, and in order to establish a functional basis, we first want to develop a purely software-based solution. The following steps shall therefore provide a reference:

- Development of a "Secure Communication" component
  - Shall implement the **if\_OS\_Socket** interface for usage by other components
  - Respective **write/recvfrom** calls need to be implemented
    - On write access
      - Encrypt the message received from the application via the **if\_OS\_Socket** interface
      - Forward it to the network stack using the **if\_OS\_Socket** interface
    - On read access
      - Decrypt the message received from the network stack via the **if\_OS\_Socket** interface
      - Forward it to the application using the **if\_OS\_Socket** interface
  - Keys shall be stored and loaded via the TRENTOS KeyStore API
  - Encryption/Decryption of messages shall be done via the TRENTOS Crypto API
  - As entropy source, the TRENTOS **EntropySource** component shall be used
- Remove the following parts from the existing setup
  - TRENTOS Filesystem API utilization from **TestApp**
  - TRENTOS **StorageServer** component utilization
  - TRENTOS **RPi\_SPI\_Flash** component utilization
- Extension of the **TestApp** component in order to
  - utilize the TRENTOS Socket API
  - make use of the "Secure Communication" component via the **if\_OS\_Socket** interface
  - reuse the existing networking capabilities (networking as client) introduced as part of Homework Task 5
  - communicate with a remote server
    - the server implementation is provided to you in form of a Python script (**srv.py**), bundled with respective instructions (**README.md**)
    - key exchange is done out-of-band, meaning the application's public key is configured on the server side
    - additionally the server's public key is stored within the application
    - communication is done exclusively encrypted, without any form of handshake
  - visualize the decrypted data on the console

The final software-based solution shall look like the following:





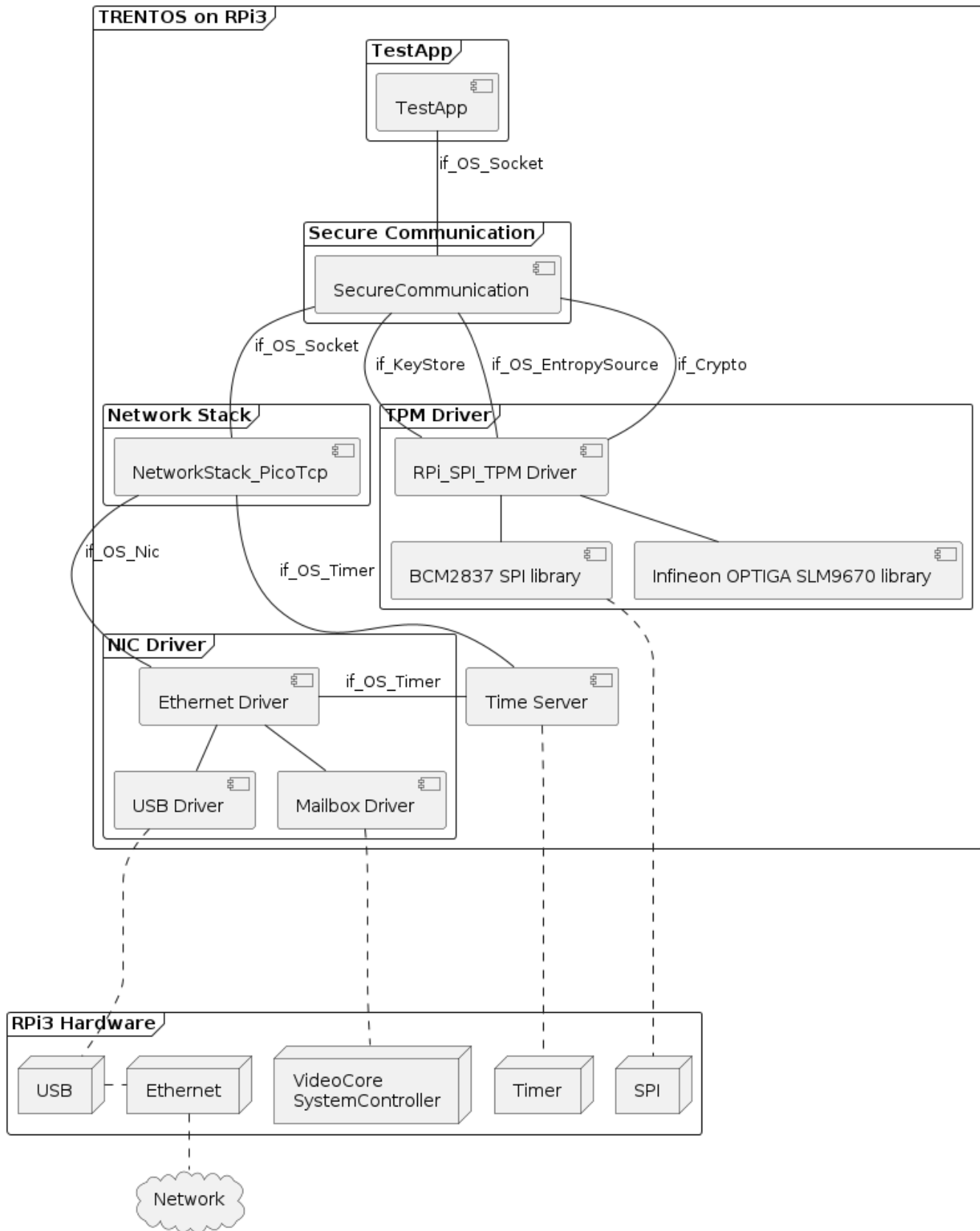
## RPi Secure Communication - Architecture

### TPM-based Solution

For a successful driver development and integration of the LetsTrust TPM breakout board into our system, the following steps shall provide a reference:

1. As already mentioned before, we can only support one SPI-based peripheral for now on the RPi 3 B+. Hence, analogous to the software-based solution, we have to remove the
  - a. TRENTOS **RPi\_SPI\_Flash** component utilization
  - b. TRENTOS Filesystem API utilization from **TestApp**
  - c. TRENTOS **StorageServer** component utilization
2. Afterwards, we have to provide the correct wiring of the SPI-based TPM breakout board to the RPi 3 B+. Please use the wiring of the SPI NOR Flash described in the TRENTOS Handbook, chapter "TRENTOS RPi\_SPI\_Flash" as reference.
3. Nevertheless, the driver internals still have to be adapted to the new Infineon OPTIGA SLB9670 TPM 2.0 based hardware device. As we want to provide support for a SPI-based peripheral, please have a look at the TRENTOS **RPi\_SPI\_Flash** component (see TRENTOS Handbook, chapter "TRENTOS RPi\_SPI\_Flash"). It can be considered as a starting point regarding the driver implementation for SPI-based peripherals in general, as it already provides the required abstractions and support for the respective low-level mechanics:
  - a. the GPIO handling (see **bcm2837\_gpio.h** and **bcm2837\_gpio.c**)
  - b. the SPI protocol (see **bcm2837\_spi.h** and **bcm2837\_spi.c**)
4. On top of the GPIO and SPI abstractions, we then have to provide the actual peripheral-specific functionality. Within the TRENTOS **RPi\_SPI\_Flash** component, this is wrapped into a peripheral-specific library, e.g. here for using a flash storage on top of the SPI protocol (see **spiflash.h** and **spiflash.c**). We can do the same for our SPI-based TPM peripheral (see section "Potential resources" below for more information). It is recommended to have a look at the open-source TCG TPM2 Software Stack and the open-source wolfTPM library.
5. Finally, we have to create a TRENTOS specific wrapper on top of this which represents the actual "driver" component within the OS (for the TRENTOS **RPi\_SPI\_Flash** component: **RPi\_SPI\_Flash.c** and **RPi\_SPI\_Flash.camkes**).
6. Examine both the mbedTLS library (which we use within the TRENTOS Crypto API) and the Infineon OPTIGA SLB9670 TPM 2.0 to select a cryptographic algorithm that is supported by both variants, e.g. the RSA2048.
7. Development of the "TPM Driver" component
  - a. Expose the TPM's RNG via the **if\_OS\_EntropySource** interface
  - b. Provide your own interface (**if\_KeyStore**) for loading/storing keys
  - c. Provide your own interface (**if\_Crypto**) for generating keys and encryption/decryption of data
8. Step by step migrate the software-based solution to the TPM-based solution
  - a. Replace the TRENTOS **EntropySource** component by connecting to the **if\_OS\_EntropySource** interface of the "TPM Driver"
  - b. Replace the utilization of the TRENTOS KeyStore API by using respective TPM-based functions offered via the **if\_KeyStore** interface of the "TPM Driver"
  - c. Replace the utilization of the TRENTOS Crypto API by using respective TPM-based functions offered via the **if\_Crypto** interface of the "TPM Driver"

The final TPM-based solution shall look like the following:



**RPi TPM-based Secure Communication - Architecture**

## Evaluation

We want to evaluate both solutions (software-based and TPM-based) and compare them according to their performance. Therefore we want to measure the performance of the following operations:

1. Generation of the private/public key pair
2. Encryption of outgoing messages
3. Decryption of incoming messages

## Potential Resources

The SPI-based TPM breakout board we will utilize for the project task is based on the Infineon OPTIGA SLB9670 TPM 2.0. Please have a look at the following resources, which provide helpful information and source code examples for driver/library development based on this specific chipset. These resources can be used as a starting point for implementing (or rather porting) your own driver/library to TRENTOS. Note: don't remove any license related information (e.g. GPL statements) and document from which sources you have taken code or code snippets for your own driver!

- <https://letstrust.de/>
- <https://www.infineon.com/cms/de/product/security-smart-card-solutions/optiga-embedded-security-solutions/optiga-tpm/slm-9670/>
- <https://www.infineon.com/cms/de/product/security-smart-card-solutions/optiga-embedded-security-solutions/optiga-tpm/slb-9670vq2.0/#!/documents>
- [https://www.infineon.com/dgdl/Infineon-SLM%209670-DataSheet-v01\\_00-EN.pdf?fileId=5546d46269e1c019016a21819cc80c93](https://www.infineon.com/dgdl/Infineon-SLM%209670-DataSheet-v01_00-EN.pdf?fileId=5546d46269e1c019016a21819cc80c93)
- [https://www.infineon.com/dgdl/Infineon-product-brief-optiga-tpm9670-PB-v01\\_00-EN.pdf?fileId=5546d462696dbf1201697c89f4bb4789](https://www.infineon.com/dgdl/Infineon-product-brief-optiga-tpm9670-PB-v01_00-EN.pdf?fileId=5546d462696dbf1201697c89f4bb4789)
- [https://www.infineon.com/dgdl/Infineon-App-Note-SLx9670-TPM2.0\\_Embedded\\_RPi\\_DI\\_SLx-ApplicationNotes-v01\\_03-EN.pdf?fileId=5546d46267c74c9a01684b96e69f5d7b](https://www.infineon.com/dgdl/Infineon-App-Note-SLx9670-TPM2.0_Embedded_RPi_DI_SLx-ApplicationNotes-v01_03-EN.pdf?fileId=5546d46267c74c9a01684b96e69f5d7b)
- <https://www.infineon.com/cms/de/product/evaluation-boards/iridium-slm-9670-tpm2.0/>
- <https://github.com/tpm2-software/tpm2-tss>
- <https://github.com/wolfssl/wolfTPM>
- [https://ubs\\_csse.gitlab.io/secu\\_os/tutorials/tpm\\_rpi.html](https://ubs_csse.gitlab.io/secu_os/tutorials/tpm_rpi.html)

## Things to watch out for

- By default, the LetsTrust TPM uses SPI 0, chip select 1
- The RPi GPIO Pin 18 needs to be pulled high
- No symmetric encryption is supported by our TPM